Data Quality and Validation in ETL

SECTION A - Theoretical Questions

Question 1: Define Data Quality in the context of ETL pipelines. Why is it more than just data cleaning?

- Definition: Data quality in ETL refers to how closely data meets the criteria for accuracy, completeness, consistency, reliability, and timeliness required for operational and analytical use.
- Why it's more than data cleaning: Data cleaning only fixes errors *after* they happen (like removing nulls or typos). Data quality encompasses a broader strategy—including validation rules, structural governance, schema enforcement, monitoring pipelines proactively, and ensuring data is fit for business purpose from extraction through to loading.

Question 2: Explain why poor data quality leads to misleading dashboards and incorrect decisions.

- Flawed Analytics: Inaccurate or incomplete data leads to incorrect aggregation metrics (e.g., distorted sales totals or miscalculated customer counts).
- Misleading Dashboards: Executives and stakeholders relying on visual dashboards will spot false trends, misjudge customer behavior, or misallocate budgets based on corrupted metrics.
- Cascading Errors ("Garbage In, Garbage Out"): Downstream machine learning models trained on bad data will yield inaccurate predictions, causing poor business strategies and financial losses.

Question 3: What is duplicate data? Explain three causes in ETL pipelines.

- Duplicate Data Definition: Identical or near-identical records appearing multiple times within a dataset where each record should logically be unique.
- Three Causes in ETL Pipelines:
  Multiple Data Sources: Pulling records from different systems (e.g., CRM and billing logs) that contain overlapping customer records without proper deduplication keys.
  Pipeline Retries/Job Failures: If an ETL batch job fails halfway and restarts, network timeouts or lack of idempotency can cause the same batch of records to be loaded twice.
  User Entry Errors: Customers or agents submitting forms multiple times (e.g., clicking a submit button twice due to slow response times).

Question 4: Differentiate between exact, partial, and fuzzy duplicates.

- Exact Duplicates: Records where every single column and attribute matches completely row-for-row.
- Partial Duplicates: Records that share a unique business key (such as `Customer_ID` and `Product_ID`) but have discrepancies or nulls in other non-key columns.
- Fuzzy Duplicates: Records that represent the exact same real-world entity but look different due to typos, formatting variations, or abbreviations (e.g., "Rahul Mehta" vs. "R. Mehta" or "Mumbai" vs. "Mumbay").

Question 5: Why should data validation be performed during transformation rather than after loading?

- Catching Errors Early: Validating data during transformation prevents corrupted, malformed, or invalid data from entering the production data warehouse or data lake.
- Cost Efficiency: Fixing errors during the staging/transformation phase is computationally cheaper and faster than running cleanup scripts or rolling back full tables in a production environment.
- Maintaining Warehouse Integrity: It ensures that constraints (like primary keys and referential integrity) are not violated upon loading.

Question 6: Explain how business rules help in validating data accuracy. Give an example.

- Role of Business Rules: They act as logical constraints dictated by company policy or domain logic to check if data values make real-world sense before storing them.
- Example: A business rule stating that `Quantity > 0` and `Txn_Amount > 0` validates data accuracy. If a transaction has a quantity of `Null` or a negative amount, the rule catches it as an anomaly, preventing invalid financial records from being processed.

SECTION B - Practical Questions

Question 7: Write an SQL query on to list all duplicate keys and their counts using the business key (Customer_ID + Product_ID + Txn_Date + Txn_Amount)

Answer: Explanation :

Business Key Definition: A business key is a combination of natural attributes that uniquely identify a real-world business event or transaction—in this case, the combination of `Customer_ID`, `Product_ID`, `Txn_Date`, and `Txn_Amount`.

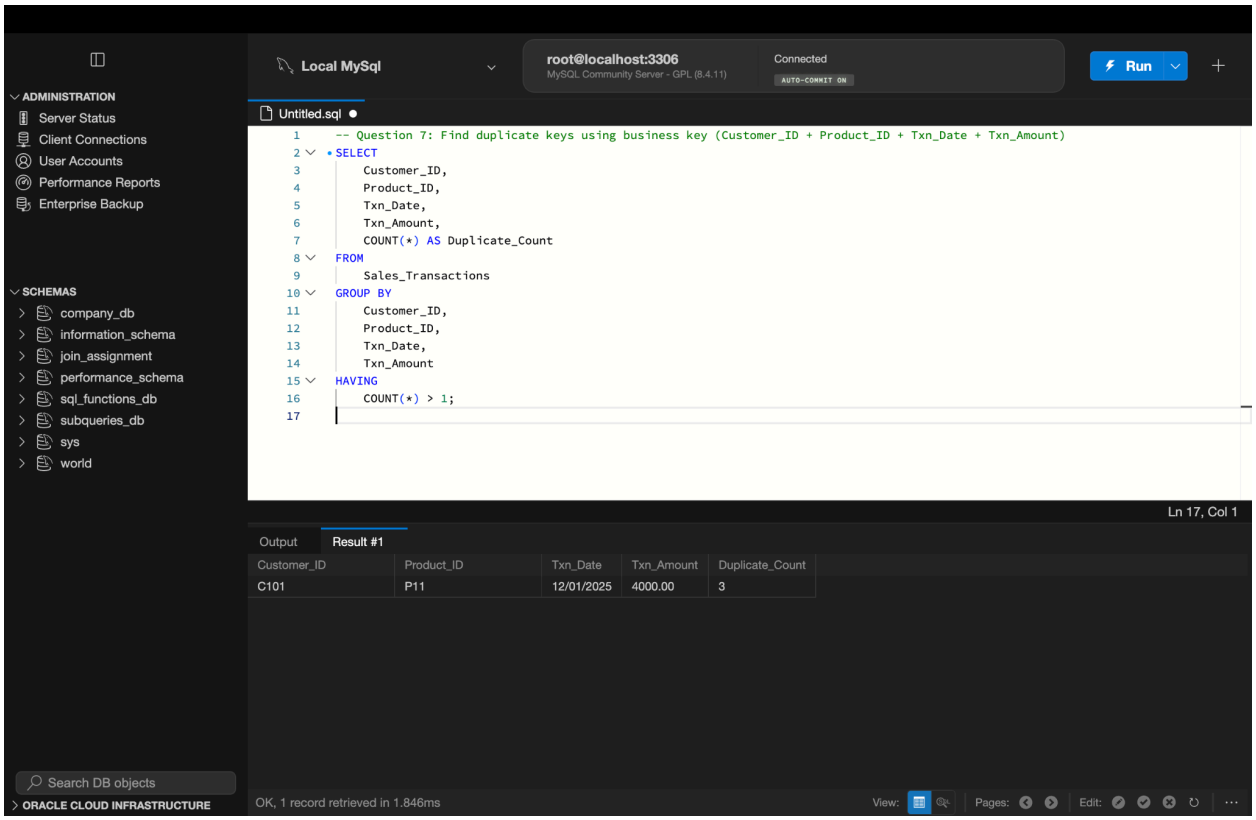
- Query Logic:
  The `GROUP BY` clause aggregates rows that share the exact same values across all

specified business key columns.
The `COUNT(*)` function tallies how many times each combination appears in the dataset.
The `HAVING COUNT(*) > 1` clause filters the results to display *only* those rows that appear more than once, effectively isolating the duplicate transactions.

: SQL Query

```
SELECT
    Customer_ID,
    Product_ID,
    Txn_Date,
    Txn_Amount,
    COUNT(*) AS Duplicate_Count
FROM
    Sales_Transactions
GROUP BY
    Customer_ID,
    Product_ID,
    Txn_Date,
    Txn_Amount
HAVING
    COUNT(*) > 1;
```

Practical proof :



Question 8: Enforcing Referential Integrity Assume the following table:
Identify Violating Customer_ID values:
Identify Sales_Transactions.Customer_ID values that violate referential integrity when joined with Customers_Master and write a query to detect such violations.

Answer :. Explanation :

Referential Integrity Definition: A database constraint mechanism that ensures relationships between tables remain consistent. It dictates that a foreign key value in a child table (like `Sales_Transactions.Customer_ID`) must always correspond to a valid primary key in the parent reference table (`Customers_Master.CustomerID`).

- Analysis & Identification:
  The `Customers_Master` table contains valid customer IDs: `C101`, `C102`, `C103`, and `C104`.
  The `Sales_Transactions` table includes transactions associated with customer IDs `C105` (Transaction 206) and `C106` (Transaction 207).
  Because C105 and C106 do not exist in the `Customers_Master` table, they violate referential integrity.
- Query Logic:
  A `LEFT JOIN` is performed from `Sales_Transactions` (

t

-) to `Customers_Master` (

c

-) on the customer identifier.
  The `WHERE c.CustomerID IS NULL` condition filters out all successfully matched rows, isolating only the orphan records where a transaction points to a non-existent master customer record.

SQL Query :

```
SELECT DISTINCT t.Customer_ID
FROM Sales_Transactions t
LEFT JOIN Customers_Master c ON t.Customer_ID = c.CustomerID
WHERE c.CustomerID IS NULL;
```

Practical proof :

